Insegnare a programmare nell’era dell’AI generativa e dei coding agent

Un documento di contesto, con le evidenze

> ✅ **Verificato sulle fonti primarie il 21 settembre 2026.** Ogni cifra è stata ricontrollata sul documento originale (paper, PDF ufficiale, pagina del report) e non sulle sintesi giornalistiche. Dove un numero è risultato errato, non verificabile o attribuito alla fonte sbagliata, il testo lo segnala con ⚠️ o 🚨. Il registro completo della verifica è in `audit-fonti.md`.

Scritto il 15 settembre 2026. È pensato per essere letto dall’inizio alla fine: parte dal mercato del lavoro, passa per la tecnologia, arriva alla ricerca sull’apprendimento e alla didattica, e chiude con gli argomenti. Ogni affermazione con un numero rimanda alla fonte; dove la fonte è debole lo dico.

---

1. Da dove nasce la domanda

La domanda “ha ancora senso insegnare a programmare?” non è nuova. Nel 2012 Code.org lanciava “Everybody should learn to code” e Obama registrava video per l’Hour of Code. Nel febbraio 2024 Jensen Huang, CEO di Nvidia, diceva al World Government Summit che “nessuno dovrebbe più imparare a programmare” perché il linguaggio di programmazione del futuro è quello umano. Nel maggio 2025 Dario Amodei, CEO di Anthropic, dichiarava ad Axios che l’AI avrebbe potuto eliminare metà dei lavori white-collar entry-level entro cinque anni e che “entro il 2026 l’AI scriverà essenzialmente tutto il codice” (Fortune, Axios). Huang gli ha risposto che “dissente praticamente su tutto”. Nell’ottobre 2025 Code.org ha trasformato l’Hour of Code in **Hour of AI** (THE Journal). Nel febbraio 2026 IBM annunciava di **triplicare** le assunzioni entry-level, “sì, anche per sviluppatori software” (Axios).

In quattordici anni il pendolo ha fatto un giro completo e mezzo. Vale la pena guardare cosa dicono i dati invece delle dichiarazioni, perché i dati raccontano una storia più interessante di entrambi gli estremi: né “tutti devono programmare” né “nessuno deve più farlo”, ma **qualcosa di specifico è cambiato in cosa significa saper programmare, e chi insegna deve capire esattamente cosa**.

---

2. Il mercato del lavoro: cosa sta succedendo davvero

2.1 Il canarino nella miniera

Lo studio più solido sugli effetti occupazionali dell’AI generativa è **“Canaries in the Coal Mine? Six Facts about the Recent Employment Effects of Artificial Intelligence”** di Erik Brynjolfsson, Bharat Chandar e Ruyu Chen dello Stanford Digital Economy Lab (pagina della pubblicazione, PDF agosto 2026). Non usa sondaggi né annunci di lavoro, ma i dati di busta paga di ADP, che gestisce gli stipendi

di oltre 25 milioni di lavoratori americani. È il primo studio con dati amministrativi ad alta frequenza su questo tema.

La versione di novembre 2025 trovava che i lavoratori tra i 22 e i 25 anni nelle occupazioni più esposte all'AI avevano subito un **calo relativo dell'occupazione del 16%**, controllando per gli shock a livello di singola azienda. ⚠ *Gli autori hanno de-enfatizzato questa stima nella versione 2026 — "Earlier versions of this study headlined regression estimates… We now emphasize the simpler descriptive divergence" — quindi non va più usata.* Per gli sviluppatori software, sempre nella versione di novembre 2025 e con dati fino a settembre 2025: *"employment for software developers aged 22-25 declined nearly 20% compared to its peak in late 2022"* — quasi 20%, e rispetto al **picco**, non alla fine del 2022. ⚠ *La "crescita del 6–12% per gli sviluppatori più anziani" che circola è errata su due fronti: il paper dice 6–9%, e si riferisce ai lavoratori più anziani nelle occupazioni esposte in generale, non agli sviluppatori software. Nella versione di agosto 2026 il caso degli sviluppatori è stato spostato nell'appendice, senza percentuale: non attribuire il "quasi 20%" all'update 2026.*

L'aggiornamento dell'agosto 2026 aggiunge tre cose importanti. La prima è che il gap si è allargato: i giovani nelle occupazioni esposte sono ora **circa il 19% sotto** dove sarebbero se avessero seguito i coetanei in occupazioni meno esposte (era il 15% a luglio 2025). Questa versione fornisce anche i numeri **in livelli**, che sono i più chiari da usare perché non passano da alcun modello statistico: *"employment of 22–25-year-olds in the two most exposed quintiles fell about 11% between November 2022 and June 2026, while employment of the same age group in the three least-exposed quintiles grew about 10%."* La seconda è che l'aggiustamento avviene "principalmente attraverso la **riduzione delle assunzioni** di giovani, non attraverso un aumento dei licenziamenti": non si licenziano i junior, semplicemente non li si assume più. La terza, e forse la più utile per chi insegna, è la distinzione tra automazione e aumento: "i cali sono concentrati nelle occupazioni in cui l'uso dell'AI tende ad **automatizzare** i compiti umani. Nelle occupazioni in cui l'AI è usata più per **complementare** i lavoratori, l'occupazione è piatta o in crescita, in particolare tra i lavoratori più esperti". Gli autori spiegano il fenomeno con la differenza tra **conoscenza codificata** — formale, documentata, quella dei libri, dei tutorial e di Stack Overflow — che l'AI sostituisce bene, e **conoscenza tacita**, quella che viene dall'esperienza, che l'AI non replica. I giovani hanno molta della prima e poca della seconda; per questo sono i canarini. L'effetto persiste anche escludendo le aziende tecnologiche e le occupazioni informatiche, quindi non è un artefatto del ciclo dei licenziamenti tech.

Vanno però conosciuti i **limiti che gli autori stessi dichiarano nell'abstract**, perché sono espliciti: questi sono *"early, descriptive indicators — canaries in the coal mine — rather than causal estimates"*. Inoltre le stime **si attenuano controllando per il livello di istruzione** (che, ammettono, può essere tanto una spiegazione alternativa quanto il canale stesso attraverso cui l'AI agisce); esistono **trend divergenti già prima di ChatGPT**, in particolare attorno al COVID; e la divergenza è **più marcata nel campione ADP** che nei benchmark delle indagini nazionali. Non è un dato causale: è un canarino che ha smesso di cantare.

Questo studio da solo cambia i termini della domanda. Non è "l'AI sostituisce i programmatori": i programmatori esperti stanno crescendo. È "l'AI sta togliendo i primi pioli della scala". E una scala senza i primi pioli è un problema che si manifesta con anni di ritardo, quando i senior di oggi invecchiano e non c'è nessuno cresciuto abbastanza per sostituirli.

2.2 Come la vivono i neolaureati

Handshake, la principale piattaforma di placement universitario negli USA, ha pubblicato uno spotlight sulla Class of 2026 in Computer Science (vedi anche il rapporto generale). Il **70% dei laureandi in CS** è almeno "somewhat pessimistic" sulla propria carriera, contro il 61% della media dei laureandi; solo il **14% è ottimista**. Tra i pessimisti, il **64% indica l'AI generativa come fattore** (47% nella media). Il dato più interessante però è un altro: il **42% dei laureandi CS usa l'AI generativa ogni giorno**, contro il 24% degli altri. Sono contemporaneamente i più preoccupati dall'AI e i suoi utenti più intensi. È una posizione psicologicamente difficile, e chi forma persone in ingresso nel settore deve tenerne conto.

Sull'entità del calo delle assunzioni di neolaureati nelle Big Tech circolano molti numeri (7% delle assunzioni nel 2025 secondo SignalFire, –25% rispetto al 2023, Google e Meta a metà dei livelli del 2021). Sono plausibili e coerenti con Stanford, ma provengono da rapporti di venture capital e blog di settore; li citerei con prudenza. Ancora più cautela per numeri virali come "–67% degli annunci entry-level" o "54% delle aziende ha smesso di assumere junior": non ho trovato la fonte primaria.

2.3 Le iscrizioni a informatica calano

Per la prima volta dopo oltre quindici anni di crescita ininterrotta, le iscrizioni ai corsi di informatica negli Stati Uniti sono in calo. Secondo il National Student Clearinghouse Research Center, riportato da Fortune (6 agosto 2026), le iscrizioni a Computer & Information Sciences sono scese dell'**8,4% nella primavera 2026** nelle università quadriennali e dell'**11,2%** nei college biennali; le iscrizioni graduate sono scese del **14%** nell'autunno 2025. ⚠️ *Il "–3,6% nell'autunno 2025" riguarda specificamente le istituzioni "primarily associate-baccalaureate", non l'undergraduate quadriennale in generale.* Il contrasto che conta più della percentuale: questo calo avviene **mentre le iscrizioni universitarie complessive crescono dell'1,3%**, tanto che NSC parla di *"un'anomalia nelle tendenze di iscrizione più ampie"*. E c'è un dato che quasi nessuno cita: la **persistenza al secondo semestre** degli studenti di informatica è la **più bassa tra i dieci corsi di laurea principali (85,0%)** — non solo se ne iscrivono meno, ma quelli che si iscrivono abbandonano di più (NSC, 11 agosto 2026). Il "62% dei programmi di computing con iscrizioni in calo" viene da Built In e non l'ho verificato su CRA; Forrester prevede un ulteriore –20%.

La reazione dei campus è istruttiva: l'AI si sposta *fuori* dal dipartimento di informatica. La University of Florida offre più di 200 corsi legati all'AI in tutti i suoi 16 college e il 71% dei laureati della primavera 2026 ne ha seguito almeno uno; Ohio State ha reso obbligatoria la "AI fluency" nella propria disciplina per tutti gli studenti a partire dalla classe del 2029; Colby College ha un istituto da 30 milioni di dollari per integrare l'AI in ogni materia. Come dice David Watts di Colby nel pezzo di Fortune, gli studenti non stanno abbandonando la tecnologia, stanno cercando "un approccio interdisciplinare all'AI", e "il modo in cui applichi quella conoscenza sta cambiando".

C'è un paradosso doppio qui. Gli studenti fuggono da CS proprio mentre ogni disciplina diventa "computing più qualcosa". E il calo delle iscrizioni del 2026 è la carenza di senior del 2031.

2.4 Il contro-trend: chi assume junior *grazie* all'AI

Nel febbraio 2026 Nickle LaMoreaux, Chief Human Resources Officer di IBM, ha annunciato che l'azienda avrebbe **triplicato le assunzioni entry-level negli Stati Uniti** nel 2026, e lo ha detto in modo volutamente provocatorio: "stiamo triplicando le assunzioni entry-level, e sì, questo vale per gli sviluppatori software e per tutti quei lavori che ci dicono che l'AI può fare" (Axios, CIO). Ma il ruolo è stato ridisegnato: i junior passano meno tempo sul coding di routine, che l'AI gestisce, e più tempo con i clienti e a **validare gli output dell'AI**. Un junior può generare cinque soluzioni possibili nel tempo in cui prima ne scriveva una; il lavoro del senior si sposta verso "valutazione, riconoscimento di pattern, e insegnare ai giovani a esercitare il giudizio". La motivazione dichiarata è di lungo periodo: tagliare i junior riduce i costi oggi ma tra tre-cinque anni ti lascia senza mid-level, e dovrai comprarli dai concorrenti a caro prezzo. Anche Cognizant ha annunciato aumenti di assunzioni entry-level, e la crescita delle assunzioni junior si sta spostando verso aziende SaaS di media dimensione, agenzie, e organizzazioni non-tech con team di sviluppo interni, oltre a sanità e pubblica amministrazione.

Il junior, insomma, non sparisce: cambia job description. Dal "scrivi questo CRUD" al "genera cinque soluzioni, dimmi quale è giusta e perché". E questo è esattamente il tipo di competenza che una formazione ben progettata può costruire.

2.5 Le competenze richieste

Il quadro macro viene dal **WEF Future of Jobs Report 2025** (sezione skills): il 39% delle competenze core dei lavoratori cambierà entro il 2030; l'**analytical thinking** resta la competenza più richiesta (sette aziende su dieci la considerano essenziale); le competenze in crescita più rapida sono AI e big data (il 90% dei datori di lavoro prevede un aumento della domanda), poi reti e cybersecurity, poi alfabetizzazione tecnologica. Il rapporto insiste che i lavori in crescita richiedono la **combinazione** di competenze tecniche e umane, non l'una o l'altra.

Sul lato più specifico, i dati Lightcast indicano che gli annunci che richiedono competenze AI pagano il **28% in più** (circa 18.000 dollari l'anno negli USA), e con almeno due competenze AI il **43% in più** (sintesi su aiforswes). Circa un annuncio su venti menziona l'AI a inizio 2026, ma nei ruoli data e analytics si arriva al 45%. Alcune fonti commerciali segnalano che gli annunci per software engineer sono cresciuti del 30% nel 2026, ai livelli più alti da tre anni (Metaintro); è una fonte da verificare, ma è coerente con il quadro generale: la domanda si è spostata, non è scomparsa. Le competenze citate più spesso sono i **fondamentali di software engineering** più le competenze specifiche dell'ecosistema AI: RAG, sistemi agentici, MLOps, sicurezza dell'AI, data engineering.

2.6 L'Italia

I dati italiani puntano nella stessa direzione, con una sfumatura in più: il problema italiano non è un eccesso di informatici, è che le aziende non trovano quelli che cercano.

Il **Sistema Informativo Excelsior di Unioncamere** (comunicato) riporta che nel 2025 le imprese cercavano 670.000 laureati e 120.000 diplomati ITS, e che **il 50,9% dei laureati e il 57,3% dei diplomati ITS sono di difficile reperimento** (mismatch complessivo 47%): più di un'offerta su due per un laureato rischia di restare scoperta. ⚠️ *Il dato ">55% per Ingegneria, Informatica, Matematica e Fisica" che circola non è nel comunicato Unioncamere (26 novembre 2025, dati 2025): viene da una sintesi giornalistica. Potrebbe essere nel rapporto completo, ma non l'ho verificato — non attribuirlo a*

Excelsior finché non lo trovi. Il Rapporto AlmaLaurea 2026, su 335.000 laureati del 2025 in 81 atenei, colloca Informatica e tecnologie ICT tra i gruppi con le **più alte chance occupazionali** a un anno dalla laurea, subito dopo le professioni sanitarie. Gli **Osservatori del Politecnico di Milano** (Artificial Intelligence) rilevano che il 78% delle organizzazioni fatica ad assumere, con la difficoltà principale nel trovare competenze tecniche in AI, Big Data e Cybersecurity, e che **il 76% degli annunci per white-collar ad alta qualificazione richiede anche competenze AI**.

🧯 **Un dato molto citato ma che non ho potuto verificare.** La ricerca **"Age of Augmentation" di Randstad** (2026) esiste ed è basata su 35,8 milioni di annunci in 11 paesi tra il 2021 e il 2026, di cui **427.768 in Italia**. Secondo la stampa italiana (Quotidiano.net, Forme), negli annunci italiani sarebbe cresciuta la domanda di **pensiero critico (+303%)**, creatività (+156%), problem solving (+133%), intelligenza emotiva (+120%), **"AI auditing" (+122%)** e quality assurance (+101%). ⚠️ *Nessuno di questi numeri è confermato da materiale Randstad primario: circolano solo su testate italiane che riportano la stessa sintesi. Sul materiale Randstad compaiono cifre diverse — prompt engineering +403%, promozioni 3,5× più veloci (non 2,5×). Da verificare sul report originale prima di usarli in pubblico: è esattamente il tipo di numero che una redazione trascrive male.* Un'inchiesta di Today.it dell'estate 2026 riporta che gli annunci per profili junior sono circa il 4% contro il 18% per i senior; non ho potuto verificare la fonte primaria di questo dato.

Messo insieme: il mercato italiano chiede sviluppatori che sappiano *giudicare* — fare auditing, QA, esercitare pensiero critico — e non li trova. Questo è precisamente il profilo che la formazione dovrebbe produrre.

---

3. La tecnologia: cosa fanno davvero gli strumenti e cosa producono

3.1 Adozione e fiducia vanno in direzioni opposte

Il **Stack Overflow Developer Survey 2025** (risultati, analisi sul trust gap) fotografa un paradosso: l'**84%** degli sviluppatori usa o pianifica di usare strumenti AI (era il 76% nel 2024), un record, ma — nelle parole del report — *"more developers actively distrust the accuracy of AI tools (46%) than trust it (33%)"*. E sono gli **sviluppatori più esperti** a fidarsi meno: solo il **2,5%** dichiara "highly trust", il **20,7%** "highly distrust". ⚠️ *La cifra "29% si fida, −11 punti rispetto al 2024" che circola è sbagliata: il 29,6% è solo la fetta "somewhat trust", scambiata per il totale; il totale è 32,7%. E il confronto col 2024 non compare nel report.* La frustrazione più citata (66%) sono le soluzioni "**quasi giuste, ma non del tutto**", e il 45% dice che debuggare codice generato dall'AI richiede più tempo. Il 52% non usa agenti o usa solo strumenti semplici. La survey 2026, aperta a giugno con il sottotitolo "for human developers only", ha nuove sezioni su agenti e ROI e dovrebbe uscire in autunno: vale la pena controllarla.

Il **GitHub Octoverse 2025** (post ufficiale) aggiunge il lato dell'ingresso: **36 milioni di nuovi sviluppatori** in un anno, più di uno al secondo, e **l'80% di loro usa Copilot entro la prima settimana**. TypeScript ha superato Python e JavaScript come linguaggio più usato, spinto dalla generazione AI; Python è cresciuto del 48,8% in contributor ed è alla base di metà dei nuovi repository AI.

Accostare questi due dati è già un argomento: i nuovi arrivati adottano lo strumento dal primo giorno, gli esperti sono quelli che se ne fidano meno. Chi ha ragione? Probabilmente entrambi, ed è proprio questa la questione didattica.

3.2 Come vengono usati gli agenti

Anthropic analizza l'uso reale di Claude. La differenza che conta è **tra strumenti**: nel report ***AI's Impact on Software Development*** (28 aprile 2025, su 500.000 conversazioni di coding raccolte tra il 6 e il 13 aprile 2025), su **Claude Code** — l'agente — il **79% delle conversazioni è classificato come "automation"** (l'AI esegue il compito), contro il **49%** di Claude.ai. ⚠️ *Sono dati vecchi di diciassette mesi, su un Claude Code appena lanciato: il confronto fra i due strumenti nello stesso momento regge, il valore assoluto "oggi" no. E soprattutto: questo dato non è nell'Economic Index di gennaio 2026, dove Claude Code non compare affatto.* Il rapporto di gennaio 2026 dà invece, per Claude.ai a novembre 2025, **52% augmentation / 45% automation**, con l'API enterprise a tre quarti di automation. Il coding resta circa un terzo del traffico consumer e circa metà di quello API.

Collegando questo a Stanford: gli agenti spingono verso l'automazione, e l'automazione è dove l'occupazione dei giovani cala. Non è colpa dello strumento, ma il default dello strumento va nella direzione sbagliata per chi sta imparando.

3.3 Produttività: percezione e realtà

Lo studio più discusso del 2025 è quello di **METR** (blog): un RCT con **16 sviluppatori** open-source esperti su **246 issue** dei *propri* repository, con e senza strumenti AI. Il campione è piccolo e va dichiarato. Risultato: con l'AI i task hanno richiesto il **19% di tempo in più** (intervallo di confidenza tra +2% e +39%). Il dato memorabile è un altro: gli sviluppatori si aspettavano un'accelerazione del 24%, e *dopo* aver sperimentato il rallentamento credevano ancora che l'AI li avesse resi il 20% più veloci. Nel follow-up del febbraio 2026 METR riporta, sui partecipanti tornati, una stima di accelerazione del 18%, ma con un intervallo (–38%/+9% di rallentamento) che include lo zero e con effetti di selezione che rendono il dato, nelle loro parole, "very weak evidence"; hanno cambiato il design dell'esperimento. Gli autori stessi elencano cosa lo studio **non** dimostra, a partire da *"non affermiamo che i sistemi AI non accelerino molti o la maggior parte degli sviluppatori"*. La lezione robusta non è "l'AI rallenta" ma **"la percezione della propria produttività con l'AI è inaffidabile"**, e questo vale a maggior ragione per uno studente che deve giudicare se sta imparando.

Il **DORA 2025 "State of AI-assisted Software Development"** di Google (rapporto, capabilities model) è più ottimista ma con una condizione: l'AI è un **amplificatore**, che potenzia le organizzazioni sane e amplifica le disfunzioni di quelle malate. Il valore non sta nello strumento ma nelle pratiche attorno, e le sette capability che identificano — una posizione chiara sull'AI, dati di qualità, dati interni accessibili all'AI, version control maturo, lavoro in piccoli batch, focus sull'utente, piattaforme interne di qualità — sono tutte competenze di ingegneria del software, nessuna è "saper scrivere prompt".

3.4 Qualità e sicurezza del codice generato

Veracode ha testato più di 100 modelli su 80 task di coding (rapporto 2025, aggiornamento primavera 2026). Il codice generato funziona, ma nel rapporto 2025 **fallisce i test di sicurezza nel 45% dei casi**; **Java al 72%**, C# 45%, JavaScript 43%, Python 38%. Sul cross-site scripting (CWE-80) i modelli

falliscono nell'**86%** dei campioni rilevanti. Il dato che pesa di più: **non è migliorato**. ⚠️ *Precisione necessaria: l'aggiornamento della primavera 2026 ragiona in tassi di superamento anziché di fallimento, e i valori oscillano tra il 45% e il 55% a seconda del taglio. Quindi non dire "45% invariato" — le due misure non sono sovrapponibili. La formulazione che regge su entrambi i report è: "circa la metà del codice generato non supera i test di sicurezza, e questo numero non si muove da un anno e mezzo". XSS e log injection restano le categorie peggiori (pass rate 13–15%).* La Cloud Security Alliance parla di "security debt del vibe coding" e di un aumento delle CVE legate a codice generato (CSA).

Sul volume, il dato "+75% di codice committato rispetto al 2022" attribuito a **GitClear** ⚠️ *non è verificabile su fonti GitClear primarie: viene da una sintesi Dark Reading.* I numeri GitClear che ho verificato sono altri, e raccontano meglio la stessa storia: blocchi di codice duplicato **8×** nel 2024, copy/paste nello stesso commit **+41%**, duplicazione di blocchi **+81%**, costrutti che mascherano gli errori **+47%**, churn a due settimane **+15%**, e soprattutto **refactoring –70%** e **manutenzione del codice legacy –74%** rispetto al 2022. Non è che si scrive più codice: **si smette di sistemarlo.**

🚫 **Una fonte che ho scartato, e perché conviene saperlo.** Circola molto uno studio di **Apiiro** secondo cui gli sviluppatori assistiti dall'AI committano 3–4 volte più spesso e generano dieci volte più problemi di sicurezza, con una scomposizione seducente: errori di sintassi –76% e logic bug –60%, ma privilege escalation +322% e difetti di design architetturale +153%. È ripreso da The Register e SiliconANGLE, e racconta perfettamente la tesi di questo documento — l'AI elimina gli errori che faceva un junior e moltiplica quelli che serve un senior per vedere.

Non lo uso. Apiiro è un vendor di *application security posture management*: vende la piattaforma che individua esattamente i problemi che la sua ricerca descrive. Lo studio è un post sul blog aziendale, senza peer review né metodologia pubblicata; i "security findings" sono l'output del loro stesso prodotto; e soprattutto **non dichiarano come abbiano classificato uno "sviluppatore assistito da AI"**, che non è un dettaglio ma la variabile su cui poggia l'intero confronto — se chi adotta l'AI per primo è già diverso (più giovane, su progetti greenfield, in team che spingono di più), lo studio non misura l'AI, misura chi la adotta. The Register aggiunge un'osservazione tagliente: Apiiro conta solo il tempo per *generare* il codice, non quello per sistemarne i difetti.

Il criterio generale che ne deriva vale per tutto questo documento: quasi tutta la ricerca sul tema viene da aziende che hanno qualcosa da vendere, e la domanda da farsi non è "hanno un interesse?" ma **"lo posso controllare, o devo crederlo?"**. Veracode è un vendor come Apiiro, ma pubblica un *benchmark* — 100+ modelli su 80 task standardizzati — che chiunque può rifare. Un numero che conferma perfettamente quello che vuoi dire merita più scetticismo, non meno.

Il filo rosso è chiaro: il codice si produce a costo quasi zero, ma **qualcuno deve saperlo leggere**, e la lettura critica del codice è una competenza che si acquisisce scrivendone.

3.5 La visione di chi costruisce gli strumenti

Andrej Karpathy, che nel 2025 ha coniato "vibe coding", nel maggio 2026 al Sequoia Ascent ha ribattezzato il suo discorso "From Vibe Coding to Agentic Engineering" (le sue note, trascrizione). Il suo frame è **Software 3.0**: si programmano gli LLM tramite prompt, contesto, strumenti, esempi, memoria; "la programmazione passa dallo scrivere codice allo **scrivere contesto**". La distinzione che fa è utile per un talk: il **vibe coding alza il pavimento** (chiunque può costruire software), l'**agentic engineering alza il soffitto** (specifiche, review, test, gestione dei permessi per guidare agenti

probabilistici e fallibili). Si può delegare all'agente la memorizzazione delle API, ma le decisioni sulla "forma del sistema" — identità, confini di sicurezza, integrità dei dati — restano umane. La frase che chiude: **"understanding cannot be outsourced"**.

Questo è coerente con i dati di Stanford (l'AI sostituisce la conoscenza codificata, non quella tacita), con Veracode (il 45% va letto da qualcuno) e con DORA (contano le pratiche, non lo strumento). Chi costruisce gli strumenti e chi misura i loro effetti dicono la stessa cosa.

---

4. La ricerca sull'apprendimento: cosa succede a chi impara con l'AI

Questa è la parte decisiva. Se il lavoro richiede persone capaci di capire e validare codice, e se questa capacità si costruisce scrivendo codice, allora la domanda diventa: **cosa succede all'apprendimento quando l'AI scrive al posto tuo?** Ci sono ormai diversi esperimenti controllati, e la risposta è sorprendentemente coerente.

4.1 Lo studio di Anthropic sulla formazione delle competenze

Nel gennaio 2026 Anthropic ha pubblicato **"How AI assistance impacts the formation of coding skills"** (pagina della ricerca), un RCT con 52 sviluppatori, per lo più junior, con almeno un anno di Python settimanale e familiarità con l'AI, che dovevano imparare una libreria nuova (Trio, per la programmazione asincrona) implementando due feature. Metà con l'assistente AI, metà a mano. Poi un quiz sui concetti appena usati.

Il gruppo con l'AI ha ottenuto in media il **50%** contro il **67%** di chi aveva scritto a mano — "l'equivalente di quasi due voti in meno" (Cohen's d = 0,738, p = 0,01). Il gruppo AI aveva finito circa due minuti prima, differenza non significativa. Il gap più grande era sulle domande di **debugging**.

Il risultato più utile per chi insegna è però l'analisi dei pattern d'uso. Tre pattern erano associati a punteggi sotto il 40%: la **delega totale** (far generare tutto), la **reliance progressiva** (iniziare con domande, finire delegando), e il **debugging iterativo** (chiedere all'AI di "aggiustare" invece di spiegare). Tre pattern erano associati a punteggi sopra il 65%: **generare e poi chiedere spiegazioni**, chiedere **codice e spiegazione insieme**, e la **"conceptual inquiry"** — fare solo domande concettuali e risolvere gli errori da soli — che era anche il secondo gruppo più veloce in assoluto. Gli autori concludono che "i guadagni di produttività possono avere come costo le competenze necessarie a validare il codice scritto dall'AI", e che "lo sforzo cognitivo — e persino il restare dolorosamente bloccati — è probabilmente importante per la padronanza".

È rilevante che sia l'azienda che vende l'agente di coding a pubblicare questo. E il dettaglio sulla "conceptual inquiry" smonta l'idea che ci sia un trade-off obbligato tra velocità e apprendimento: chi usa l'AI per capire va veloce *e* impara. La differenza è un'abitudine, e le abitudini si insegnano.

4.2 Guardrail sì, guardrail no: l'esperimento di Wharton

"Generative AI without guardrails can harm learning: Evidence from high school mathematics" di Hamsa Bastani e colleghi, pubblicato su PNAS nel 2025 (PNAS, Knowledge at Wharton), è

l'esperimento educativo sull'AI più citato ed è fuori dalla programmazione, il che lo rende ancora più interessante come principio generale. Quasi 1.000 studenti di matematica delle superiori in Turchia divisi in tre gruppi: uno con **GPT Base** (un'interfaccia simile a ChatGPT), uno con **GPT Tutor** (lo stesso modello ma con guardrail progettati con gli insegnanti: dà suggerimenti, non risposte), e un gruppo di controllo senza tecnologia.

Durante la pratica entrambi i gruppi con l'AI andavano meglio del controllo: +48% con GPT Base, +127% con GPT Tutor. Ma all'**esame senza AI**, il gruppo GPT Base ha fatto **il 17% peggio** di chi non aveva mai avuto l'AI. Il gruppo GPT Tutor non ha subito danni. Lo stesso modello, con vincoli pedagogici, elimina il danno; senza vincoli, chi lo usa impara meno di chi non lo usa affatto.

La conclusione degli autori — l'AI "può migliorare significativamente le performance a breve termine, ma strumenti mal progettati possono danneggiare l'apprendimento a lungo termine incoraggiando l'overreliance e minando l'acquisizione di competenze" — trasforma la domanda "AI sì o no" nella domanda "AI progettata come". Ed è una domanda per chi insegna.

4.3 Il risultato ottimista: i novizi con Codex

Per onestà intellettuale va citato anche **Kazemitabaar et al.** (CHI 2023, arXiv; sintesi in Austin Henley): 69 novizi tra i 10 e i 17 anni, 45 task Python, metà con accesso a Codex. Con Codex i punteggi erano 1,8 volte più alti e i completamenti 1,15 volte, **senza cali** nei task di modifica manuale del codice; al post-test dopo una settimana, il gruppo Codex era leggermente migliore, senza significatività statistica. Non tutta l'evidenza è negativa, quindi. La differenza con Anthropic e Bastani sta probabilmente nel design dei task (molto strutturati, con esercizi manuali intercalati) e nella popolazione. Ma questo rafforza, non indebolisce, la tesi: il *come* fa la differenza.

4.4 L'AI allarga il divario

"The Widening Gap: The Benefits and Harms of Generative AI for Novice Programmers" di James Prather e colleghi (ICER 2024, ACM, arXiv) è uno studio **qualitativo su 21 studenti**, condotto attraverso 21 sessioni di laboratorio con osservazione diretta, interviste ed **eye-tracking**. Il punto di partenza è la letteratura sulla **metacognizione**: i principianti fanno fatica perché formano modelli sbagliati del problema, hanno un "falso senso di progresso" dopo aver testato una soluzione, non sanno quando fermarsi a riflettere. Il risultato: tutte queste difficoltà **restano, vengono amplificate dall'AI, e ne emergono di nuove**. E soprattutto: chi **accelerava** usava la GenAI per scrivere codice che sapeva già di voler scrivere e sapeva ignorare i suggerimenti sbagliati; chi **arrancava** finiva con *"dissonanza cognitiva sulla propria capacità di risolvere problemi"*, convinto di aver fatto meglio di quanto avesse fatto, e con quella che gli autori chiamano un'**"illusione di competenza"**. L'AI **allarga il divario**. ⚠️ *La formulazione "studenti con voti e autoefficacia più alti", che circola nelle sintesi, non compare nell'abstract: la frase da citare è "illusione di competenza", che è testuale.*

Per chi lavora nel reskill questo è un dato da tenere davanti agli occhi: gli adulti in riqualificazione sono spesso proprio il gruppo a bassa autoefficacia che l'AI rischia di lasciare indietro, mentre sembra aiutarli.

4.5 Gli effetti cognitivi più generali

Due studi fuori dalla programmazione completano il quadro. Il **Microsoft Research + Carnegie Mellon** presentato a CHI 2025 (pubblicazione) ha intervistato 319 knowledge worker su 936 casi d'uso reali: **più fiducia nell'AI, meno pensiero critico**; più fiducia in sé stessi, più pensiero critico. Chi usa l'AI si sposta dal "fare" al "verificare" — ma solo se ha la competenza per verificare. Lo studio del **MIT Media Lab "Your Brain on ChatGPT"** (pagina, arXiv) ha usato l'EEG su 54 partecipanti che scrivevano saggi con LLM, con motore di ricerca o senza aiuti: chi usava l'LLM mostrava la connettività cerebrale più debole, minor senso di ownership del testo, difficoltà a citarlo. Gli autori parlano di **"cognitive debt"**. È un preprint con un campione piccolo, da usare come metafora più che come prova, ma la metafora è buona: come il debito tecnico, il debito cognitivo si accumula in silenzio e si paga tutto insieme.

Il paper **"The Effortless Trap: Productive Struggle, AI, and the Illusion of Learning"** (Brcic e Frljic, 2026, arXiv) tira le fila: la contrapposizione "permettere o vietare" è un falso dilemma; la vera domanda progettuale è **dove collocare** l'AI nel processo di apprendimento. Usata male, l'AI sostituisce il lavoro cognitivo che l'apprendimento richiede e lascia "un'illusione di apprendimento: un senso di padronanza che crolla di fronte al compito senza aiuti". La regola diagnostica, testuale: *"if letting AI in makes the task feel effortless, it is in the wrong place"*. Il loro framework in sei passi (Prime, Probe, Point, Attach, Strengthen, Test) protegge il **primo tentativo difficile** e la **verifica finale senza aiuti**, e tratta la facilità come un segnale d'allarme. Il fondamento teorico è la letteratura sulle "desirable difficulties" di Bjork: ciò che rende la pratica facile la rende spesso inefficace.

---

5. La didattica: cosa sta cambiando nel modo di insegnare

5.1 La comunità di ricerca ha già cambiato idea

Nel 2025 il **Dagstuhl Seminar 25311 "Generative AI in Programming Education"** (pagina), organizzato da Brett Becker (scomparso nel frattempo), Michelle Craig, Paul Denny, Natalie Kiesler e James Prather — cioè il nucleo della ricerca mondiale sulla didattica della programmazione — ha messo nero su bianco che "l'approccio tradizionale di far scrivere agli studenti molti piccoli programmi controllati da strumenti di valutazione automatica è stato **probabilmente reso obsoleto** dall'AI generativa", dato che strumenti facili da usare risolvono i problemi introduttivi con un prompt in linguaggio naturale. Il report è in Dagstuhl Reports vol. 15 n. 7 (2026). Lo stesso gruppo ha prodotto il working group report ITiCSE **"Beyond the Hype"** (arXiv), una rassegna completa di ricerca, pratiche e strumenti.

Nel febbraio 2027 si terrà il **Dagstuhl Seminar 27062 "What Should We Teach Computer Science Students in the Age of AI?"** (pagina), organizzato da Leinonen, Prather, Renzella e Vassar. La questione si è spostata dalla pedagogia al **curriculum**: "i modelli educativi e curricolari focalizzati sulla programmazione dai primi principi e sulla padronanza della sintassi potrebbero non preparare più adeguatamente i laureati"; le linee guida internazionali **CS2023 precedono questo cambiamento** e non bastano. Le domande aperte: quali fondamenti restano essenziali e quali diventano obsoleti; quali competenze emergenti (ragionamento data-driven, comprensione dell'infrastruttura AI) vanno prioritizzate; come valutare la comprensione profonda quando "l'AI genera soluzioni perfette agli

esercizi standard". Che la comunità mondiale stia ridiscutendo il curriculum a questo livello è di per sé una notizia.

5.2 Gli approcci concreti

Nel frattempo sono emersi approcci con evidenza sperimentale.

I **Prompt Problems** di Paul Denny e colleghi (SIGCSE 2024, Interactions with Prompt Problems, tool Promptly) rovesciano l'esercizio classico: lo studente riceve un problema in forma **visuale** — apposta, per impedire il copia-incolla nel modello: i partecipanti ricevono *"visual representations of a coding problem, without any text that they could copy and paste into their prompt"* — e deve scrivere un **prompt in linguaggio naturale** tale che l'LLM generi codice che passa i test. Allena la specificazione precisa, la decomposizione del problema, la lettura del codice generato, il debugging del prompt. Lo studio esplorativo su 36 studenti osserva due comportamenti ricorrenti: all'inizio i prompt sono **vaghi**, senza abbastanza dettaglio per risolvere il problema; e quando le cose non funzionano, gli studenti **aggiungono messaggi di chiarimento invece di ristrutturare il prompt originale** — lo stesso pattern di "debug iterativo" che nell'RCT di Anthropic sta sotto il 40% di comprensione. Due studi indipendenti, stesso comportamento disfunzionale. Lo shift dichiarato dagli autori: "dalla scrittura di codice alla **lettura, comprensione e valutazione** del codice generato dagli LLM", più la nuova competenza di risolvere problemi costruendo buoni prompt. E costruire un buon prompt, in fondo, richiede esattamente ciò che programmare ha sempre richiesto: capire il problema abbastanza bene da descriverlo senza ambiguità.

Le domande **"Explain in Plain English"** (EiPE), una tecnica classica di valutazione della comprensione del codice, hanno trovato una seconda vita: in **"Explaining Code with a Purpose"** (ITiCSE 2024) la spiegazione dello studente viene usata come prompt e la sua correttezza è verificata dal codice che l'LLM genera da essa. Un solo esercizio allena comprensione e prompting, con valutazione automatica (Code Generation Based Grading).

I **Knowledge Markers** (Christina Maria Mayr, arXiv, aprile 2026) — che è una **proposta di design curricolare** e non uno studio con risultati misurati, come il paper stesso dichiara ("without claiming measured learning gains") — partono da una premessa netta: "il codice funzionante **non è più un indicatore affidabile di comprensione**". Propongono di etichettare ogni unità didattica secondo il tipo di conoscenza che allena — **(A) Application**, l'implementazione; **(S) Structure**, concetti e modelli mentali; **(P) Procedure**, metodi sistematici, decisione, verifica — così da decidere in modo esplicito dove l'AI è ammessa e dove no.

Un RCT del febbraio 2026 (arXiv) mostra che passare da una "policy sull'AI" a un'**istruzione esplicita e scalabile al prompting** in un corso CS1 produce effetti misurabili. L'experience report **DURA** (arXiv) — Demystify, Use, Reflect, Assess — descrive un CS2 in cui prima si smonta come funziona un LLM, poi lo si usa, poi si riflette, poi si valuta. Studi qualitativi su come gli studenti sbagliano e debuggano nei task in linguaggio naturale (arXiv) e sulle traiettorie prompt per prompt (arXiv) stanno costruendo la base empirica.

Il modello di riferimento per il "tutor con guardrail" è il **CS50 di Harvard** (Teaching CS50 with AI, SIGCSE 2024): dal 2023 un "duck" AI che non dà soluzioni ma guida, pensato per "approssimare un rapporto 1:1 docente-studente", deployato su migliaia di studenti online e in aula, percepito come "un tutor personale". È esattamente il GPT Tutor di Bastani applicato alla programmazione.

Fuori dalle università, **Code.org** ha trasformato l'Hour of Code in Hour of AI (THE Journal, The 74); Hadi Partovi precisa che l'obiettivo non è mai stato produrre software engineer ma cittadini che capiscono come funzionano le cose, e che il coding resta al centro con l'AI intrecciata. In India, IIT Delhi integra i code generator nel *primo* corso di programmazione dal 2025–26 con enfasi sull'uso responsabile.

5.3 I temi che ricorrono

Se si guarda a tutto questo insieme, emergono alcuni principi ricorrenti. Il primo è l'**inversione della tassonomia di Bloom**: "creare" è diventato economico, "valutare" e "analizzare" sono il collo di bottiglia; il curriculum classico va dal basso verso l'alto (sintassi, scrittura, poi lettura, poi design), quello nuovo deve allenare lettura, critica e specifica molto prima. Il secondo è **comprehension-first**: leggere, spiegare, trovare l'errore, refactorare, prima o insieme allo scrivere. Il terzo è la **specificazione come competenza primaria**: il prompt è una specifica, e scrivere buone specifiche richiede di saper decomporre; era il vero contenuto del programmare da sempre. Il quarto è la **fatica produttiva protetta**: momenti senza AI progettati esplicitamente — il primo tentativo, la verifica finale — non come punizione ma come palestra. Il quinto è la **metacognizione esplicita**: insegnare a riconoscere il falso senso di progresso che l'AI amplifica. Il sesto è la scelta dello **strumento con guardrail**: la differenza tra +127% e −17% (Bastani). Il settimo è la **valutazione**: se il codice funzionante non prova più nulla, servono colloqui, spiegazioni, code review dal vivo, "difendi questa pull request".

---

6. Gli argomenti: come tenere insieme tutto questo

6.1 Naur: il programma è la teoria

Il frame più elegante per tutto il discorso viene da un testo del 1985, **"Programming as Theory Building"** di Peter Naur. Il programma, dice Naur, non è il codice: è la *teoria* che i programmatori hanno in testa su come il codice rispecchia il mondo e perché è fatto così. Il codice senza la teoria è morto: non lo si può modificare in modo sensato, e i tentativi di "documentare" la teoria per trasmetterla falliscono sempre. Il codice generato dall'AI è codice **senza teoria**, a meno che qualcuno la costruisca leggendolo. Questo collega tutto: Stanford (l'AI sostituisce la conoscenza codificata, non quella tacita — e la teoria di Naur è conoscenza tacita), Veracode (qualcuno deve leggere il 45%), Anthropic (chi delega non costruisce la teoria), Karpathy (understanding cannot be outsourced).

6.2 Le analogie storiche, e dove zoppicano

L'analogia con i **compilatori** è la più usata: negli anni '50 "i veri programmatori scrivono in assembly", poi arrivò Fortran, e nessuno rimpiange l'assembly, ma i migliori sanno cosa fa il compilatore. Un nuovo livello di astrazione non è la fine della disciplina. Dove zoppica: il compilatore è deterministico e corretto, l'LLM no. È la differenza tra un'**astrazione** e una **delega**. **Dijkstra** lo aveva già detto nel 1978 in "On the foolishness of 'natural language programming'": la precisione del linguaggio formale non è un ostacolo, è lo strumento del pensiero. Si può giocare con l'ironia — Dijkstra aveva torto, o aveva ragione e stiamo per scoprirlo? — e collegarlo a Karpathy: "writing context" è ancora scrivere

specifiche precise. L'analogia con le **calcolatrici** in matematica è comunicativa ma va maneggiata: la calcolatrice non sbaglia il 45% delle volte.

6.3 Jevons, e la scala senza pioli

Il **paradosso di Jevons** — quando il costo di una risorsa crolla, il consumo esplode — è stato citato esplicitamente da Satya Nadella nel 2025 a proposito del software. Se il costo di produrre software crolla, la domanda di software esplode, e con essa la quantità di software da capire, mantenere, mettere in sicurezza. Ma la domanda di *chi* sappia farlo si scontra con il problema della scala senza pioli: la seniority si costruisce facendo il lavoro da junior, e se si automatizza il lavoro da junior, da dove vengono i senior del 2031? Charity Majors lo aveva scritto nel 2024 in "Generative AI is not going to build your engineering team for you". IBM lo ha capito e per questo triplica i junior.

6.4 Automazione o aumento: il formatore è chi insegna la seconda

Il frame di Stanford — la stessa tecnologia, usata per automatizzare, produce disoccupazione giovanile; usata per aumentare, produce crescita — è forse il più utile per chi fa formazione professionale. Non è una scelta dello strumento: l'Anthropic Economic Index mostra che l'agente spinge al 79% verso l'automazione. È una scelta dell'utente, e dipende da abitudini che si possono insegnare: i pattern "conceptual inquiry" di Anthropic, il GPT Tutor di Bastani, i Prompt Problems di Denny, la fatica protetta dell'Effortless Trap sono tutti modi di insegnare a usare l'AI per aumentare sé stessi invece che per sostituirsi. E il mercato italiano sta pagando esattamente per questo: il **76%** degli annunci per white-collar ad alta qualificazione richiede competenze AI, e il **78%** delle organizzazioni fatica ad assumere proprio su AI, dati e sicurezza (Osservatori PoliMi). *(I numeri Randstad sul pensiero critico, spesso citati qui, sono da verificare: vedi §2.6.)*

6.5 I contro-argomenti, per non sembrare un insegnante che difende il posto

Vale la pena anticiparli. "Kazemitabaar ha trovato che i novizi con Codex non peggiorano" — vero, ma con task strutturati e ragazzi di 10–17 anni; su adulti e task realistici l'evidenza va nell'altra direzione, e comunque conferma che il design conta. "METR 2026 dice che ora l'AI accelera" — METR stessa dice che è evidenza debole e ha cambiato il design; e velocità non è apprendimento. "Il mercato junior è morto, formarli è inutile" — IBM, Cognizant, Excelsior (più del 55% introvabili in informatica), Randstad: il junior cambia forma, non scompare, e la carenza di senior del 2031 si combatte formando junior nel 2026. "Basta l'AI literacy, non serve programmare" — ma l'AI literacy per chi deve *validare* codice **è** programmazione: comprensione, debugging, sicurezza. Ohio State la rende obbligatoria per tutti; questo rende il programmatore che sa validare ancora più distintivo, non meno.

---

7. In sintesi

Il mercato non chiede meno programmatori: chiede programmatori diversi, e non li trova. L'occupazione cala solo per i giovani e solo dove l'AI automatizza; cresce dove l'AI aumenta e dove c'è esperienza. Gli strumenti producono codice a costo quasi zero e con vulnerabilità nel 45% dei casi; chi li conosce meglio se ne fida meno; chi entra nel mestiere li usa dal primo giorno. Gli

esperimenti controllati dicono che chi impara delegando all'AI capisce meno — due voti in meno, il 17% peggio all'esame — ma che lo stesso strumento, usato per capire o progettato per non dare risposte, non fa danni e a volte accelera. La comunità di ricerca sulla didattica ha dichiarato obsoleto il modello classico e sta ridisegnando il curriculum; gli approcci nuovi allenano lettura, specifica, valutazione e fatica protetta.

Ha ancora senso insegnare a programmare? Ha senso più di prima, a condizione di insegnare la cosa giusta: non a *produrre* codice, che ormai è gratis, ma a costruire la teoria che rende il codice vivo, e a usare l'AI in modo che questa teoria si formi invece di atrofizzarsi. È una competenza tacita, si costruisce con la fatica, e nessuno la può delegare. È, letteralmente, l'ultima cosa che non si può outsourcare.

Fonti citate

Mercato del lavoro - Brynjolfsson, Chandar, Chen, *Canaries in the Coal Mine?* — pubblicazione, update agosto 2026, PDF - Handshake, *Class of 2026: Computer Science majors* — spotlight, outlook - Fortune, *Computer science enrollment is plunging* (6 ago 2026) — link; Built In — link - IBM triplica le assunzioni entry-level — Axios, CIO - WEF, *Future of Jobs Report 2025* — skills outlook - Lightcast via aiforswes — link; Metaintro — link - Amodei vs Huang — Fortune, Axios

Italia - Unioncamere / Excelsior — comunicato - AlmaLaurea, *Rapporto 2026* — link - Osservatori Politecnico di Milano, *Artificial Intelligence* — link - Randstad, *Age of Augmentation* 2026 — Quotidiano.net, Forme - Today.it, inchiesta primo lavoro e AI — link

Tecnologia - Stack Overflow Developer Survey 2025 — risultati, trust gap, survey 2026 - GitHub Octoverse 2025 — link - Anthropic Economic Index — gennaio 2026, marzo 2026 - METR — studio 2025, update febbraio 2026 - DORA 2025 — rapporto, AI Capabilities Model - Veracode — 2025, primavera 2026; CSA — link; GitClear via Dark Reading — link - Karpathy, Sequoia Ascent 2026 — note, trascrizione

Apprendimento - Anthropic, *How AI assistance impacts the formation of coding skills* — link - Bastani et al., *Generative AI without guardrails can harm learning*, PNAS 2025 — PNAS, Wharton - Kazemitabaar et al., CHI 2023 — arXiv; Henley — link - Prather et al., *The Widening Gap*, ICER 2024 — ACM, arXiv - Lee et al. (Microsoft/CMU), CHI 2025 — link - MIT Media Lab, *Your Brain on ChatGPT* — pagina, arXiv - Brcic, Frljic, *The Effortless Trap* — arXiv

Didattica - Dagstuhl 25311 — link; Dagstuhl 27062 — link - Prather et al., *Beyond the Hype* (ITiCSE WG 2025) — arXiv - Denny et al., *Prompt Problems* — SIGCSE 2024, Interactions, Promptly - *Explaining Code with a Purpose* — arXiv; *Code Generation Based Grading* — arXiv - *Knowledge Markers* — arXiv - RCT prompting instruction in CS1 — arXiv; DURA — arXiv - *Teaching CS50 with AI* — PDF - Code.org, Hour of AI — THE Journal, The 74